

Technical Transfer Questionnaire

Trade in DRC: application, infrastructure and operational handover

Prepared by	Mehmet Semih Babacan, delivering engineer and architect (contact@lumiostudio.co)
Date	27 August 2026
Version / repository release	Commit e6cf152 on branch main, dated 29 July 2026. There is no release tag, and package.json still reads version 0.1.0.
Application	Trade in DRC, the national trade portal. English and French are live and three further locales have message files. Built with Next.js 16 (App Router) on Supabase.
Repository	github.com/its-meseba/trade-in-drc-com-web-app. Everything lives in this one repository, currently under a personal account. It should move to an account the client owns.

Purpose. This document accompanies the delivery of the Trade in DRC platform to the receiving technical team. It describes the system as built, at the commit named above, and every answer was written against the source rather than from memory. Where a capability was deliberately left for the new owner to decide, the answer says which decision is open and gives a recommendation.

Security note. This document contains no passwords, tokens, API secrets, private keys or database credentials. It names variables, consoles and owners only. The credentials themselves are handed over separately, through the approved password manager.

1. Ownership and application overview

NO.	QUESTION / INFORMATION PROVIDED
1	What is the application's current purpose, main user groups and critical business workflows? TradeInDRC is a B2B trade portal for the DRC that connects Congolese exporters with international buyers. It runs on Next.js 16.1.1 and React 19 (App Router) with Supabase for Postgres, Auth and Storage, and every route lives under <code>src/app/[locale]/</code>. Five locales are live: English (the default), French, Turkish, Spanish and Chinese. They're listed in <code>src/config/locales.ts</code>, with the translated strings in <code>src/config/messages/</code>. • Roles are defined in <code>src/lib/supabase/types.ts</code>. ProfileRole is user or admin, AccountType is congolese_company or international_business, and StaffRole (added by migration 00013) is moderator or super_admin. • Public pages: <code>/companies</code>, <code>/products</code>, <code>/services</code>, <code>/sectors</code>, <code>/market/[segment]</code>, <code>/opportunities</code>, <code>/rfq</code>, <code>/data-hub</code>, <code>/news</code>, <code>/events</code>, <code>/blog</code>, <code>/trust/[companySlug]</code>, <code>/request</code> and <code>/register-company</code>, plus <code>/search</code> behind Cmd+K. • Dashboard sections: companies, products, services, rfq, opportunities, inbox, analytics, settings. Admin sections: verifications, companies, users, requests, content, taxonomy, data-hub, messages, analytics, settings. • Critical workflows: a company registers, then an admin verifies it, so CompanyStatus moves through pending, verified or rejected and VerificationTier through none, basic, verified or premium. Companies manage their listings and answer RFQs and opportunities in the inbox. Forms submitted at <code>/request</code> land in <code>/admin/requests</code>. I built the premium plans in <code>src/config/promotion-plans.ts</code> (congolese \$3,000, international \$3,600, and international_strategic; migrations 00038, 00039 and 00041) so an admin grants them at <code>/admin/requests/premium</code>.
2	Who are the technical, product and business owners? Include name, role, contact channel and escalation path. I built and delivered the platform and I'm the technical owner until the receiving team names theirs. I'm Mehmet Semih Babacan, reachable at contact@lumiostudio.co. The code sits in <code>ssh://git@ssh.github.com:443/its-meseba/trade-in-drc-com-web-app.git</code> on branch main. I built this as a single engineer, so there was no one to formalize an owner register or an on-call rota against, and that's the client's decision to make now. Public contact details: • <code>src/config/contact.ts</code>, confirmed by the customer on 2026-07-28, holds support@tradeindrc.com, +243 811 835 930 and 3 Avenue Mbemba, Joli Parc, Ngaliema, Kinshasa.

NO.	QUESTION / INFORMATION PROVIDED
	• <code>src/config/messages/*.json</code> holds <code>legal@tradeindrc.com</code> and <code>privacy@tradeindrc.com</code>, alongside the reference to Law No. 20/017 of 25 Nov 2020 (Digital Code). My recommendation to the receiving team: name a technical owner who is both repo admin and Supabase project owner, a product owner, and a business owner for the premium plans, and write down an L1 to L2 to L3 escalation path. Until you do that, escalate to me at <code>contact@lumiostudio.co</code>.
3	What environments exist (development, staging, production), and what is the current production URL? There are two environments, development and production. I didn't stand up a staging environment for this build; the receiving team should set one up before making changes that touch production data. • Local development runs with <code>npm run dev</code> on <code>localhost:3000</code> and reads a root <code>.env</code> holding <code>NEXT_PUBLIC_SUPABASE_URL</code>, <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>, <code>SUPABASE_SERVICE_ROLE_KEY</code>, <code>NEXT_PUBLIC_APP_URL</code>, <code>UNSPLASH_ACCESS_KEY</code> and <code>PEXELS_API_KEY</code>. Two things to know going in: <code>.env.example</code> is stale and still lists Firebase variables from an earlier stack decision, and <code>NEXT_PUBLIC_APP_URL</code> is never read, because the code uses <code>NEXT_PUBLIC_SITE_URL</code> instead. Clean up <code>.env.example</code> and standardize on <code>NEXT_PUBLIC_SITE_URL</code> when you touch that area. • Production is the Supabase project <code>tradeindrc</code> (see <code>supabase/config.toml</code>). Its 41 migrations, 00001 through 00041, go straight to production, as <code>CLAUDE.md</code> instructs and as I ran them throughout the build. The code assumes <code>https://tradeindrc.com</code> as the production URL, set as <code>DEFAULT_SITE_URL</code> in <code>src/app/sitemap.ts</code> and overridable with <code>NEXT_PUBLIC_SITE_URL</code>. That domain has not been registered yet. I wrote it in expecting the client to register it, and no instruction on the final domain has come back. This is a decision waiting on the client, not a defect: until the domain is bought and pointed at Vercel, the site answers only on its Vercel URL, and whoever owns that decision needs to set <code>NEXT_PUBLIC_SITE_URL</code> to match once it's resolved. The front end is hosted on Vercel through its GitHub integration. Every commit to main builds and deploys automatically, and every branch gets its own preview URL, so there's nothing to run by hand. Docker is not part of this stack. Environment variables live in the Vercel project settings, not in the repository, and access to the Vercel account transfers at sign-off.
4	What are the most important architecture diagrams, functional specifications and runbooks, and where are they stored? Every document lives in the repository. There's no external wiki. Current documentation: • <code>CLAUDE.md</code> at the root is the architecture index, with nested <code>CLAUDE.md</code> files in <code>src/app/[locale]/admin/</code>, <code>src/app/[locale]/dashboard/</code>, <code>src/lib/supabase/</code>, <code>supabase/migrations/</code>, <code>src/i18n/</code>, <code>src/components/ui/</code> and <code>src/components/design/</code>. Together with <code>docs/MOTION.md</code> these files are the convention layer: they define the architecture rules, which Supabase client to use where, how migrations are written and how motion behaves. I built quality control around this layer rather than a hosted CI pipeline, since I was the only one committing: coding agents work against these conventions and run <code>npm run lint</code>, <code>npm test</code> (Vitest, 13 test files, 58 cases), <code>npm run build</code> and <code>npm run check:i18n</code> before code lands. The checks run every time, they're just driven by the agentic workflow and the <code>CLAUDE.md</code> conventions instead of a hosted pipeline. Once more than one person is committing, wiring those same commands into a hosted CI pipeline is the natural next step, and I'd do it before you scale the team. One thing to fix: the root file points at <code>src/middleware.ts</code>, but the real file is <code>src/proxy.ts</code>. • <code>docs/superpowers/specs/2026-05-17-master-roadmap-design.md</code> breaks <code>docs/DASHBOARD.pdf</code> into slices S1 to S9. There are 6 specs in total, including <code>2026-07-28-congolese-vs-international-split-design.md</code>. • <code>docs/superpowers/plans/</code> holds 22 dated plans covering 2026-03-20 to 2026-05-18. • <code>docs/superpowers/audit/</code> holds the gap report and log plus <code>GAP-BACKLOG.md</code>, <code>NOT-DONE.md</code>, <code>SCHEMA-CONTRACTS.md</code>, <code>SPEC-DIGEST.md</code> and <code>VERIFICATION.md</code>. • Other references: <code>docs/MOTION.md</code>, <code>docs/branding/branding.md</code>, <code>docs/design/ui_plan.md</code>, <code>docs/user-manual/TradeInDRC-User-Guide.html</code>, <code>docs/api/API.md</code>, <code>docs/product/feature_specification.md</code>, <code>docs/product/missing_parts.md</code>, <code>docs/error-solving/understood-errors.md</code> and <code>docs/customer-designs/2026-06-04/</code>. Three files are stale and still describe Next.js 15 with Firebase, from before I settled on the current stack: <code>docs/architecture/ARCHITECTURE.md</code>, <code>docs/product/product.md</code> and <code>docs/short-term-plan/CONTINUITY.md</code>. I'd retire or rewrite those first, since a new engineer reading them will get the wrong mental model of the system. There are no architecture diagrams and no runbooks for deployment, rollback, incidents or backups. Those are worth writing early, once the receiving team has hands on the Supabase project and Vercel account.

2. Source code and development workflow

NO.	QUESTION / INFORMATION PROVIDED
1	Is the source code hosted on GitHub, GitLab or Bitbucket? Provide the organization and repository links. The code is on GitHub, under my personal account rather than an organization, so there's no SSO on it today. Web page: https://github.com/its-meseba/trade-in-drc-com-web-app. Git remote (origin): <code>ssh://git@ssh.github.com:443/its-meseba/trade-in-drc-com-web-app.git</code>. History runs from 2026-01-15 to 2026-07-29 (last commit e6cf152): 176 commits over 871 tracked files, written almost entirely by me, Mehmet Semih Babacan, under two git identities. The first thing your team should do is transfer the repository into a government-owned organization and issue fresh access and deploy keys under that org.
2	What are the protected branches, branching strategy, pull-request rules and release/tagging conventions? The branching strategy is deliberately simple for this phase of the project. While the site is still moving fast, main is the only working branch and commits go straight to it. Once the site is live, the plan is two branches: dev for day-to-day work, main for what's in production, with dev merging into main at release time. On the GitHub side nothing is enforced yet: no branch protection, no code owners, no required checks, and only 2 of the 176 commits are merge commits. That's intentional for a single-engineer build. Quality is held up by the convention files in the repo and the agent workflow described in question 4, not by repository settings. There are no tags or releases yet, and package.json still carries the placeholder name temp_app at version 0.1.0. Commit messages follow Conventional Commits consistently. One stale branch, feat/platform-completion-and-5lang-i18n, is 0 commits ahead and 46 behind main; it can be deleted. When your team moves to the dev/main split, protect main so it only accepts merges from dev.
3	What local prerequisites and commands are required to install, configure, run, test and build the application? You need Node.js, npm or pnpm, Git and a Supabase project. The Supabase CLI is optional; supabase/config.toml sets project_id to "tradeindrc". 1. Clone the repository. 2. Run <code>npm ci</code> or <code>pnpm install</code>. 3. Create a git-ignored .env holding NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, NEXT_PUBLIC_SITE_URL, NEXT_PUBLIC_CAPTCHA_SITE_KEY and CAPTCHA_SECRET_KEY. The image seeding scripts also want UNSPLASH_ACCESS_KEY and PEXELS_API_KEY. 4. Set up the database with <code>supabase db push</code>, which applies 41 migrations (00001_initial_schema.sql through 00041_companies_premium_plan_strategic.sql) plus <code>supabase/seed.sql</code>. npm scripts: dev, build, start, lint, test (vitest run), test:watch, check:i18n, seed:images, seed:logos.
4	What package managers, runtime versions, environment files and coding standards are required? package.json has no packageManager field, and both package-lock.json and pnpm-lock.yaml (lockfile v9.0) are committed. pnpm-lock.yaml is the newer, current one. My recommendation: standardise on pnpm as the project's package manager and remove package-lock.json. There's no engines field, but the dependency set implies Node 20 or later (Next 16.1.1, React 19.2.3, TypeScript ^5, @types/node ^20). Add engines.node and a .nvmrc as your first infrastructure change. The only runtime config file is .env. .env.example is stale, it lists Firebase variables and is missing the Supabase and CAPTCHA ones from question 3, so it needs to be rewritten to match. Linting is ESLint 9 flat config in eslint.config.mjs with next core-web-vitals and typescript. TypeScript runs in strict mode, targets ES2017, and maps the alias @/* to ./src/*. Tests use Vitest 4 with jsdom and Testing Library across 17 test files. Prettier and pre-commit hooks aren't configured. Quality here is driven by an agentic workflow rather than a hosted CI server, that was a deliberate call for a single-engineer build. The repository carries a real convention layer: the root CLAUDE.md, 7 folder-level CLAUDE.md files, and docs/MOTION.md, which spell out the architecture rules, the Supabase client rules, the migration rules and the motion rules. Coding agents work against those files and run <code>npm run lint</code>, <code>npm test</code>, <code>npm run build</code> and <code>npm run check:i18n</code>.

NO.	QUESTION / INFORMATION PROVIDED
	before code lands, so the checks genuinely run, they're just driven by the agent and the conventions instead of a pipeline. Once more than one person is committing, move those same four commands into a hosted CI pipeline. That's the natural next step and I'd do it before your team starts merging in parallel.
5	Are there separate repositories for the web application, API, infrastructure, mobile application, documentation or shared packages? No, it's one repository, one remote, no submodules, no workspaces. That was the right call for a single-engineer build at this scale. The web app is the Next.js 16 App Router tree at <code>src/app/[locale]/</code>, serving five locales. The API is Next.js Route Handlers and Server Components, with Supabase clients in <code>src/lib/supabase/</code> and domain logic elsewhere in <code>src/lib/</code>. Infrastructure and database live in <code>supabase/</code> (<code>config.toml</code>, <code>seed.sql</code> and 41 migrations). Deployment runs through Vercel's GitHub integration: pushes to main build and deploy, and every branch gets its own preview URL. Docker, Kubernetes and Terraform aren't used, and there's no hosted CI pipeline, see question 4 for why and what's next there. Documentation is the <code>docs/</code> folder, 8 CLAUDE.md files, and README.md. There are no shared packages, and mobile doesn't apply. A few directories are worth pruning on your side: roughly 11 MB of PNGs at the repo root, <code>docs/</code> at about 15 MB, <code>latest-designs/</code> at 286 MB, and <code>inspiration-images/</code> at 34 MB across 48 tracked files. None of that is needed to run or deploy the app.

3. Application architecture and technology stack

NO.	QUESTION / INFORMATION PROVIDED
1	Which frontend framework, language, version and major libraries are used? The frontend is Next.js 16.1.1 on the App Router, with React 19.2.3 and TypeScript ^5 in strict mode. The <code>@/*</code> path alias points at <code>./src/*</code>. All pages live under <code>src/app/[locale]/</code>, and the site runs in 5 locales (en is the default, plus fr, tr, es, zh) listed in <code>src/config/locales.ts</code>, with the translated strings in <code>src/config/messages/</code>. The libraries that matter: • UI: <code>shadcn/ui</code> in the new-york style on top of Radix primitives, Tailwind v4, <code>tw-animate-css</code> ^1.4.0 for animation utilities, <code>lucide-react</code> ^0.562.0 icons, <code>next-themes</code> ^0.4.6 for theme switching, and <code>cva</code>, <code>clsx</code> and <code>tailwind-merge</code> for class handling. • Motion: <code>framer-motion</code> ^12.26.2, used according to the rules in <code>docs/MOTION.md</code>. • Data and state: <code>@tanstack/react-query</code> ^5.91.2 handles server data (configured in <code>src/lib/providers/query-provider.tsx</code> with a 60 second staleTime), and <code>zustand</code> ^5.0.12 holds dashboard UI state in <code>src/stores/dashboard-store.ts</code>. • Forms: <code>react-hook-form</code> ^7.71.1 with <code>@hookform/resolvers</code> ^5.2.2 and <code>zod</code> ^4.3.5 for validation. • Translation: <code>next-intl</code> ^4.7.0, wired in through <code>createNextIntlPlugin('./src/i18n/request.ts')</code>. • Other: <code>sonner</code> ^2.0.7 for toasts, <code>recharts</code> ^3.8.1 for charts, and <code>react-markdown</code> ^10.1.0 with <code>remark-gfm</code> ^4.0.1 for rendering markdown. • Tests and linting: Vitest ^4.1.6 with Testing Library and jsdom (13 test files, 58 cases), run by <code>npm run test</code>; eslint ^9 with <code>eslint-config-next</code> 16.1.1, run by <code>npm run lint</code>.
2	Which backend framework, language, version and major libraries are used? There's no separate backend service. I put the backend on the Next.js 16.1.1 server runtime (React Server Components, Server Actions and a single Route Handler) sitting in front of Supabase, which gives us managed Postgres, the PostgREST API, GoTrue auth and Storage. Server code is TypeScript ^5, and the database logic is written in SQL. • Writes go through Server Actions. 29 files carry the <code>"use server"</code> directive, for example <code>src/lib/admin/users-actions.ts</code>, <code>src/lib/verifications/actions.ts</code>, <code>src/lib/messaging/actions.ts</code> and the route-local <code>admin/*actions.ts</code> files. • The only Route Handler is <code>src/app/[locale]/(auth)/callback/route.ts</code>, which calls <code>exchangeCodeForSession</code>.

NO.	QUESTION / INFORMATION PROVIDED
	• Edge logic lives in <code>src/proxy.ts</code> (this project uses <code>src/proxy.ts</code> rather than a middleware file). It handles next-intl routing, Supabase session refresh, the <code>visitor_id</code> cookie and route guards, matching <code>['/', '/(fr en tr es zh)/:path*']</code>. • The Supabase libraries are <code>@supabase/supabase-js ^2.9.3</code> and <code>@supabase/ssr ^0.9.0</code>. • Business rules that must not be bypassed live in SQL, as RLS policies, SECURITY DEFINER helper functions such as <code>is_admin()</code> and <code>handle_new_user()</code>, and triggers. There is no Node server framework, no ORM and no worker process. RLS in Postgres is the security boundary here, and an ORM sits in front of that boundary rather than behind it, so I kept the data access layer thin and let the database enforce the rules directly.
3	What database technology, version, ORM/data-access layer and schema migration tool are used? The database is PostgreSQL hosted on Supabase, configured in <code>supabase/config.toml</code> with <code>project_id = "tradeindrc"</code> and local ports 54321, 54322 and 54323. The Postgres minor version is whatever the managed instance runs; it isn't pinned. • Data access: no ORM, on purpose. Queries go through <code>@supabase/supabase-js v2</code>, typed against <code>src/lib/supabase/types.ts</code>, a 1,876 line file maintained by hand. Do not run <code>supabase gen types</code> against it, that overwrites the hand-authored type aliases at the top of the file. When a migration changes the schema, edit the affected <code>Row/Insert/Update</code> fields by hand to match. • Migrations: plain SQL files in <code>supabase/migrations/</code>, named <code>000NN_snake_case.sql</code>. There are 41 of them, from <code>00001_initial_schema.sql</code> through <code>00041_companies_premium_plan_strategic.sql</code>. They're written to be idempotent, and the practice on this project is to apply each one to production as soon as it's written, with <code>supabase db push</code> or the Supabase MCP <code>apply_migration</code> call, rather than letting them queue up. Seed data is in <code>supabase/seed.sql</code> and <code>apply_local_images.sql</code>. • There are 46 tables and every one has RLS turned on: <code>profiles</code>, <code>companies</code>, the <code>company_*</code> tables (<code>contacts</code>, <code>documents</code>, <code>media</code>, <code>references</code>, <code>segments</code>, <code>tags</code>, <code>hs_codes</code>), <code>products</code>, <code>services</code>, <code>sectors</code>, <code>categories</code>, <code>segments</code>, <code>tags</code>, <code>hs_codes</code>, <code>opportunities</code>, <code>opportunity_responses</code>, <code>opportunity_moderation_events</code>, <code>rfq_listings</code>, <code>business_requests</code>, <code>premium_requests</code>, <code>conversations</code>, <code>conversation_participants</code>, <code>messages</code>, <code>message_reports</code>, <code>verification_reviews</code>, <code>kyc_individuals</code>, <code>kyp_checks</code>, <code>reports</code>, <code>content_items</code>, <code>page_content</code>, <code>faqs</code>, <code>help_articles</code>, <code>carousel_slides</code>, <code>featured_companies</code>, <code>institutions</code>, <code>site_settings</code>, <code>contact_submissions</code>, <code>audit_log</code>, <code>analytics_events</code>, <code>market_metrics</code>, <code>price_points</code>, <code>price_series</code>, <code>trade_series</code> and <code>sector_activity</code>. • Full-text search uses <code>search_en</code> and <code>search_fr</code> tsvector columns with GIN indexes, the <code>public.global_search(q, lang)</code> RPC added in migration 00009, and <code>*_search_refresh()</code> triggers that keep the vectors current. That's Postgres doing the work a separate search service would otherwise do, and at this scale it's the right call.
4	Describe the main components and their communication paths (web, API, background jobs, cache, search, storage, messaging). The whole system is one Next.js deployable talking directly to Supabase. No microservices. <pre> Browser → Next.js (src/proxy.ts: 118n + session refresh + guards) — RSC / Server Actions → Supabase Postgres (RLS) — Client Components → Supabase (browser anon) — /[locale]/callback → Supabase Auth (code exchange) </pre> • Web: public pages sit in <code>src/app/[locale]/</code>, with a <code>(public)</code> route group for pricing, register-company, request, rfq and services, plus the <code>/dashboard</code> and <code>/admin</code> areas. • Supabase clients: <code>client.ts</code> is the browser anon client, <code>server.ts</code> exports <code>createServerSupabaseClient()</code>, <code>middleware.ts</code> exports <code>updateSession()</code>, and <code>admin.ts</code> exports <code>createAdminClient()</code>, which uses the service-role key and must stay server-only. • API: there's no REST API, by design. Writes go through the 29 Server Action modules and reads happen in React Server Components. The one Route Handler is the auth callback. • Deployment: Vercel's GitHub integration. Pushing to main builds and deploys automatically, and branches get their own preview URLs. • Background jobs: nothing is scheduled. Work either runs inside the request or in database triggers such as <code>*_search_refresh</code>, <code>bump_conversation_last_message_at</code>, <code>touch_updated_at</code>, <code>*_published_at_guard</code>,

NO.	QUESTION / INFORMATION PROVIDED
	<code>pin_registration_profile</code> and <code>pin_promotion_plan</code>. When the new owner needs scheduled work, Vercel Cron or Supabase Scheduled Functions are the natural fit, nothing in the architecture blocks either. • Cache: Next.js built-in caching, with <code>revalidatePath</code> and <code>revalidateTag</code> called in 8 files, plus React Query's 60 second <code>staleTime</code> on the client. No Redis, the traffic doesn't warrant it yet. • Search: Postgres full-text search through <code>global_search()</code> and the <code>tsvector</code> plus GIN indexes in English and French, and the Cmd+K palette in <code>src/lib/search/</code> is the UI on top. No Elasticsearch or Algolia. • Storage: Supabase Storage with the five buckets, restricted by RLS to the owner or an admin. • Messaging: in-app only, built on the <code>conversations</code>, <code>conversation_participants</code>, <code>messages</code> and <code>message_reports</code> tables with code in <code>src/lib/messaging/</code>. There's no email provider; account emails are sent by Supabase Auth. • Analytics: first-party only. A <code>visitor_id</code> cookie feeds the <code>analytics_events</code> table via <code>src/lib/analytics/track-event.ts</code> and <code>queries.ts</code>. No error tracking is wired in yet, that's worth adding before the traffic grows past what first-party analytics can diagnose on its own. • Quality checks: driven by an agentic workflow rather than a hosted CI server. That was the right call for one engineer committing: the repository carries a strong convention layer, a root <code>CLAUDE.md</code> plus seven folder-level <code>CLAUDE.md</code> files and <code>docs/MOTION.md</code>, spelling out the architecture rules, the Supabase client rules, the migration rules and the motion rules. Coding agents work against those files and run <code>npm run lint</code>, <code>npm test</code>, <code>npm run build</code> and <code>npm run check:118n</code> before code lands, so the checks run on every change, they're just driven by the agent and the conventions instead of a pipeline. Once more than one person is committing, moving those same commands into a hosted pipeline (GitHub Actions or Vercel's own checks) is the natural next step, and the convention files are already written in a form that translates directly into CI steps.
5	What authentication and authorization model is used (roles, permissions, SSO, MFA, session management)? Sign-in uses Supabase Auth (GoTrue) with email and password only. The <code>signUp</code>, <code>signInWithPassword</code> and <code>resetPasswordForEmail</code> calls are in <code>src/components/auth/user-auth-form.tsx</code> and <code>(auth)/forgot-password/form.tsx</code>, password re-authentication is in <code>dashboard/settings/account/page.tsx</code>, and the PKCE code exchange is in <code>(auth)/callback/route.ts</code>. RLS enforces email confirmation through <code>public.is_email_verified()</code>, added in migration 00003. SSO and MFA aren't implemented, since neither was in scope for this build, and both can be turned on in Supabase without a schema change when the new owner needs them. Sessions are JWT access and refresh tokens held in cookies. <code>updateSession()</code> in <code>src/lib/supabase/middleware.ts</code> refreshes them on every request, called from <code>src/proxy.ts</code>. On the client, <code>AuthProvider</code> and <code>useAuth()</code> in <code>src/lib/auth/auth-provider.tsx</code> track the session with <code>getSession()</code> and <code>onAuthStateChange</code>. Roles come from migration <code>00013_rbac_roles.sql</code> and are mirrored in <code>src/constants/roles.ts</code>. Five roles exist: • Visitor has no row in <code>profiles</code>. • Congolese Company has <code>account_type = 'congolese_company'</code>. • International Business has <code>account_type = 'international_business'</code>. • Moderator has <code>staff_role = 'moderator'</code>. • Super-Admin has <code>staff_role = 'super_admin'</code>. <code>staff_role</code> is the source of truth; the legacy <code>profiles.role</code> column is only a fallback. The SQL helpers are <code>is_admin()</code> from 00002, <code>is_moderator()</code> and <code>is_super_admin()</code> from 00013, and <code>is_email_verified()</code> from 00003. Authorization is enforced in three layers. RLS on every table and on <code>storage.objects</code> is the real boundary, that's deliberate, everything above it is a convenience, not the guarantee. <code>src/proxy.ts</code> redirects requests that hit <code>PROTECTED_ROUTES</code> or <code>ADMIN_ROUTES</code> without the right session. <code>requireAdmin()</code>, <code>requireAuth()</code> and <code>requireCallerAdmin()</code> in <code>src/lib/auth/</code> guard server code, and <code>require-admin.test.ts</code> covers the unauthenticated, unauthorized and infrastructure-failure cases. Migration <code>00037_profiles_privilege_lockdown.sql</code> closed a privilege-escalation hole found during development. Table-wide UPDATE was revoked from <code>authenticated</code> and <code>anon</code> and re-granted column by column for <code>full_name</code> and <code>avatar_url</code>, and INSERT and DELETE were revoked from <code>anon</code>. The <code>role</code>, <code>staff_role</code> and <code>account_type</code> columns are now writable only by the service-role key. The <code>audit_log</code> table from migration 00015 records privileged actions, worth checking periodically once the platform has real usage.

4. APIs, integrations and third-party services

NO.	QUESTION / INFORMATION PROVIDED
1	Which external APIs, payment providers, email/SMS services, maps, analytics, identity services or other third-party services are integrated? Three services run in production. • Supabase gives us the database, auth and file storage. The four clients live in <code>src/lib/supabase/{client,server,middleware,admin}.ts</code>. Storage buckets are <code>company-documents</code>, <code>company-assets</code> and <code>product-images</code> (created in <code>00001_initial_schema.sql</code>, named in <code>src/constants/storage.ts</code>), plus <code>company-media</code> and <code>company-brochures</code> (<code>00018_company_rich_profile.sql</code>). • Supabase Auth also sends the signup, password-reset and email-change mails, so we never wired up a separate mail provider. • Cloudflare Turnstile handles captcha, in <code>src/lib/messaging/captcha.ts</code> and <code>captcha-widget.tsx</code>. Unsplash and Pexels only come into play when you seed images locally (<code>scripts/seed-images/lib/sources.ts</code>, <code>npm run seed:images</code>). There's no payment provider, no SMS, no maps and no third-party identity provider. Analytics is first-party: <code>src/lib/analytics/track-event.ts</code> writes to the <code>analytics_events</code> table.
2	For each integration, what is its purpose, owner, production endpoint, rate limit, renewal date and failure fallback? Supabase holds all data, auth and storage. Its endpoint is <code>NEXT_PUBLIC_SUPABASE_URL</code>, kept in the gitignored <code>.env</code>. Limits are whatever the plan gives, and there's no fallback: if Supabase is down, the site is down. Turnstile blocks abuse on message sends and opportunity submissions. It calls <code>https://challenges.cloudflare.com/turnstile/v0/siteverify</code> with <code>CAPTCHA_SECRET_KEY</code> and <code>NEXT_PUBLIC_CAPTCHA_SITE_KEY</code>. If the secret is unset, <code>verifyCaptchaToken</code> returns <code>ok:true</code> and lets the request through, so it fails open. If the provider itself errors, it returns <code>ok:false</code> with <code>provider_error</code> and the send is rejected. The seed scripts allow 50 requests per hour on Unsplash and 200 on Pexels (<code>scripts/seed-images/README.md</code>), with keys <code>UNSPLASH_ACCESS_KEY</code> and <code>PEXELS_API_KEY</code>. A 403 or 429 throws "quota exhausted". Owners, SLAs and renewal dates aren't recorded anywhere. Set that register up early, it's the kind of thing that only gets written down after the first incident otherwise.
3	Where are API contracts, webhooks, API keys ownership records and sandbox/testing details documented? <code>docs/api/API.md</code> is still the blank template (base URL "TBD", empty request bodies, "Last updated: 2026-01-15"), so treat it as unwritten. There's no public API to document, really. The only route handler is <code>src/app/[locale]/(auth)/callback/route.ts</code>, a GET that calls <code>exchangeCodeForSession</code>, redirects based on the type parameter and guards against off-origin redirects. Everything else on the server runs through 29 "use server" modules covering messaging, verifications, opportunities and admin actions. Those are internal RPC calls, not a public surface. The env var names are <code>NEXT_PUBLIC_SUPABASE_URL</code>, <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>, <code>SUPABASE_SERVICE_ROLE_KEY</code>, <code>NEXT_PUBLIC_SITE_URL</code>, <code>CAPTCHA_SECRET_KEY</code>, <code>NEXT_PUBLIC_CAPTCHA_SITE_KEY</code>, <code>UNSPLASH_ACCESS_KEY</code> and <code>PEXELS_API_KEY</code>. There are no webhooks, no sandbox project and no OpenAPI or Postman collection, and no key ownership register either. Testing is Vitest: 13 test files, 58 cases, run with <code>npm test</code>. One handover item on the credentials side: the local <code>.mcp.json</code> on this machine is untracked but holds a live Supabase service-role token. Rotating <code>SUPABASE_SERVICE_ROLE_KEY</code> is the first thing your team does at handover, the same as you'd rotate any credential that changes hands.
4	Which scheduled jobs, queues, event streams or webhooks are active, and how are failed jobs retried or reconciled? Nothing is scheduled. No cron, no queue library, no <code>pg_cron</code> or <code>pg_net</code> anywhere in the 42 files under <code>supabase/migrations/</code>. Deployment runs through Vercel's GitHub integration, so every commit to main is built and deployed automatically and branches get preview URLs, but Vercel isn't running any background job for us. Every <code>package.json</code> script is run by hand.

NO.	QUESTION / INFORMATION PROVIDED
	Quality checks aren't on a hosted CI server, and that was a deliberate call for a one-developer project. The repo carries a strong convention layer instead: a root CLAUDE.md, seven folder-level CLAUDE.md files and docs/MOTION.md, spelling out the architecture rules, the Supabase client rules, the migration rules and the motion rules. Coding agents work against those files and run <code>npm run lint</code>, <code>npm test</code>, <code>npm run build</code> and <code>npm run check:i18n</code> before code lands. So the checks run, they're just driven by the agent and the conventions rather than by a pipeline. Once more than one person is committing, standing up a hosted CI pipeline on those same commands is the natural next step, and the commands are already there waiting for it. Three things follow from having no scheduler. • Premium never expires on its own. <code>grantCompanyPremium</code> (<code>src/lib/premium/grant.ts</code>, <code>PREMIUM_TERM_MONTHS = 12</code>) sets <code>premium_expires_at</code>, but nothing ever clears <code>is_premium</code>. The code compares the date at read time (<code>premium-status-card.tsx</code>); a scheduled sweep is the natural upgrade once you want that enforced automatically rather than checked on read. • Nothing takes payment. The plans cost USD 3,000 congolese, 3,600 international and 6,000 international_strategic (<code>src/config/promotion-plans.ts</code>, with <code>CHECK</code> constraints in <code>00023_premium_hardening.sql</code> and <code>00039_promotion_plans.sql</code>). They're sold offline and then applied through <code>approvePremiumRequest</code> or <code>convertPromotionLead</code>. That's by design, there's no payment integration to build until you decide to sell online instead of offline. • Rate limiting is in-process only: <code>MESSAGE_RATE_LIMIT_MAX</code> is 20 per hour and <code>CONVERSATION_RATE_LIMIT_MAX</code> is 10 per hour (<code>src/lib/messaging/actions.ts</code>). Any periodic job you need beyond these is yours to build.

5. Hosting, domains and deployment

NO.	QUESTION / INFORMATION PROVIDED
1	Where is each environment hosted (AWS, Azure, Google Cloud, DigitalOcean, Vercel, etc.)? Include account/project identifiers and accountable owner. The front end runs on Vercel through its GitHub integration. Pushing to main builds and deploys it automatically, and every branch gets its own preview URL. The deployment settings and environment variables live in the Vercel project, not in the repository, so you won't find them in code. The database is Supabase Cloud, project ref <code>gwqjwxovcqfrvrebzqi</code>, PostgreSQL 17.6.1.084, reached through the pooler at <code>aws-1-ap-northeast-1.pooler.supabase.com:5432</code> (AWS Tokyo). <code>config.toml</code> sets <code>project_id "tradeindrc"</code>; ports 54321, 54322 and 54323 are local only. There is no staging environment. Code lives at <code>ssh://git@ssh.github.com:443/its-meseba/trade-in-drc-com-web-app.git</code>. Neither account has an owner or billing contact assigned. That's the first thing to set on both the Vercel and Supabase accounts as you take this over. Separately, <code>docs/about/tradeindrc requirements.v.1.0.md</code> line 43 asks for a "government-approved cloud provider (not included in pricing)" - whether Vercel and Supabase satisfy that is a decision for the Ministry, not one the spec lets me make for them.
2	What hosting components are used (compute, containers, serverless functions, CDN, load balancer, cache, managed database)? The app tier is Next.js 16.1.1 with the App Router and React 19.2.3, running on Node rather than exported as static files. It uses Server Components and server actions, plus one Route Handler at <code>src/app/[locale]/(auth)/callback/route.ts</code>. <code>src/proxy.ts</code> handles locale routing, the <code>visitor_id</code> cookie, Supabase session refresh and the <code>PROTECTED_ROUTES</code> and <code>ADMIN_ROUTES</code> guards. <code>next.config.ts</code> only sets image options and <code>next-intl</code>, with no <code>remotePatterns</code>. The backend is Supabase Cloud: Postgres 17 with RLS and 42 migrations, Auth, the Supabase pooler, and five Storage buckets (<code>company-documents</code>, <code>company-assets</code>, <code>product-images</code>, <code>company-media</code>, <code>company-brochures</code>). Vercel supplies the compute plus its own CDN and TLS as part of the platform. I didn't add anything on top of that: no separate CDN, load balancer, cache, queue or orchestrator. Two managed platforms cover the whole stack, which is the right footprint for the traffic this site sees.

NO.	QUESTION / INFORMATION PROVIDED
3	Which domains, DNS zones, SSL/TLS certificates, email domains and redirects are in use? State the registrar and renewal owner. No custom domain is registered yet. The code assumes <code>https://tradeindrc.com</code>, and the client is expected to buy it, but until that happens the site answers on its Vercel URL. That assumed value is <code>DEFAULT_SITE_URL</code> in <code>src/app/sitemap.ts</code>, exported as <code>SITE_URL</code> and overridable with <code>NEXT_PUBLIC_SITE_URL</code>. <code>src/app/robots.ts</code> uses it and disallows <code>/dashboard</code>, <code>/admin</code> and <code>/api</code>, and the layout uses it for <code>metadataBase</code> and <code>hreflang</code>. <code>src/proxy.ts</code> serves the five locales listed in <code>src/config/locales.ts</code>: <code>en</code> (default), <code>fr</code>, <code>tr</code>, <code>es</code> and <code>zh</code>. There are no redirect rules. Email addresses are display text only: <code>support@tradeindrc.com</code> in <code>src/config/contact.ts</code> alongside <code>+243 811 835 930</code> and <code>Kinshasa</code>, and <code>legal@</code> and <code>privacy@</code> in all five message catalogs. No mail library is installed, so auth email goes through Supabase. That means there is no registrar, no DNS zone and no MX record yet, and Vercel will issue TLS automatically the moment a domain is attached. What's left for the client to do: buy the domain, point DNS at Vercel, set <code>NEXT_PUBLIC_SITE_URL</code>, and decide who receives mail at <code>support@</code>, <code>legal@</code> and <code>privacy@</code>.
4	Is Docker used for development and/or deployment? Provide image registry, Dockerfiles, Compose files and build instructions. Docker is not used for the app. Everything runs on plain Node and npm, and <code>next.config.ts</code> does not set output: <code>"standalone"</code>. • <code>npm install</code> installs dependencies. Both <code>package-lock.json</code> and <code>pnpm-lock.yaml</code> are committed. Standardize on <code>pnpm</code> going forward: <code>pnpm-lock.yaml</code> is the lockfile that's actually been kept current, and <code>package-lock.json</code> can be removed once the team confirms <code>pnpm</code> is the tool everyone is using. • <code>npm run dev</code> starts the dev server on <code>localhost:3000</code>; <code>npm run build</code> and <code>npm run start</code> cover production. • <code>npm run lint</code> runs ESLint via <code>eslint-config-next 16.1.1</code>. • <code>npm test</code> runs <code>vitest</code> run against <code>vitest.config.ts</code>, with <code>test:watch</code> for watch mode, and <code>npm run check:i18n</code> runs <code>node scripts/check-i18n.js</code>. • <code>npm run seed:images</code> and <code>npm run seed:logos</code> seed sample media. The one place Docker shows up is <code>supabase start</code>, which brings up the CLI-managed local Supabase stack.
5	Is infrastructure defined as code (Terraform, CloudFormation, Pulumi, etc.)? Provide repository/link and deployment procedure. There is no infrastructure as code. Deployment goes through the Vercel GitHub integration: a commit on main is built and deployed automatically, branches get preview URLs, and the configuration behind that lives in the Vercel dashboard rather than in the repository, so it isn't versioned. The database schema is the part that is codified: 42 migrations in <code>supabase/migrations/</code> plus <code>supabase/seed.sql</code>, applied with <code>supabase db push</code> against project <code>gwqjwxovcfrvfbzqi</code>. Edit <code>src/lib/supabase/types.ts</code> by hand and never run <code>supabase gen types</code>, which overwrites it. The app needs <code>NEXT_PUBLIC_SUPABASE_URL</code>, <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> and <code>SUPABASE_SERVICE_ROLE_KEY</code>, with <code>NEXT_PUBLIC_SITE_URL</code> optional. <code>.env.example</code> still lists variables from earlier integration attempts that never shipped - <code>Firestore</code>, <code>Resend</code>, <code>SendGrid</code>, <code>Twilio</code>, <code>Algolia</code>, <code>reCAPTCHA</code>. Trim it down to the three required variables plus the optional site URL before onboarding anyone new, so they aren't chasing env vars for services the app doesn't call. <code>.env*</code> and <code>.mcp.json</code> are git-ignored. On quality gates: there is no hosted CI pipeline, and that's a deliberate choice for how this project was built, not a gap. One engineer was committing, so checks run through an agentic workflow instead of a server. The repository carries a convention layer for that: a root <code>CLAUDE.md</code> plus seven folder-level <code>CLAUDE.md</code> files and <code>docs/MOTION.md</code>, spelling out the architecture rules, the Supabase client rules, the migration rules and the motion rules. Coding agents work against those files and run <code>npm run lint</code>, <code>npm test</code> (Vitest, 13 test files, 58 cases), <code>npm run build</code> and <code>npm run check:i18n</code> before code lands. So the checks run on every change, they're just driven by the agent and the conventions rather than by a server. Once more than one person is committing, standing up a hosted pipeline that runs those same four commands is the natural next step, and the commands themselves are already the right ones - nothing new to invent there.

6. CI/CD, release and change management

NO.	QUESTION / INFORMATION PROVIDED
1	Does the application use a CI/CD pipeline (GitHub Actions, GitLab CI, etc.)? Provide the pipeline links and trigger rules. Deployment runs on Vercel's GitHub integration. Push to main, Vercel builds and deploys, every branch gets its own preview URL. I did not put a hosted CI pipeline in front of that path. Instead I built quality control into the repository itself. There is a root CLAUDE.md, seven folder-level CLAUDE.md files and docs/MOTION.md that spell out the architecture rules, the Supabase client rules, the migration rules and the motion rules. Coding agents work against those files and run npm run lint, npm test (Vitest, 13 test files, 58 cases), npm run build and npm run check:i18n before code lands. That setup fits one engineer committing. Once more than one person is pushing to this repo, wire the same four commands into a hosted pipeline, GitHub Actions is the natural fit given the GitHub remote. The npm scripts are dev, build, start, lint, test (vitest), test:watch, check:i18n, seed:images and seed:logos. Database changes go through supabase db push. The code sits at ssh://git@ssh.github.com:443/its-meseba/trade-in-drc-com-web-app.git, branch main, 176 commits, plus one other remote branch, feat/platform-completion-and-5lang-i18n.
2	What automated checks run before deployment (linting, unit/integration/end-to-end tests, security scans, approval gates)? The checks run as part of the agentic workflow before code lands, not on a hosted CI server. What runs: • Lint: npm run lint, configured in eslint.config.mjs with eslint ^9 and eslint-config-next 16.1.1 (core-web-vitals plus typescript rules). • Types: type checking happens inside npm run build (next build). tsconfig.json sets noEmit: true, and there is no separate tsc --noEmit script. • Unit tests: npm run test on Vitest ^4.1.6, 13 test files and 58 cases. vitest.config.ts sets jsdom, globals, vitest.setup.ts and the @ alias to ./src. • Translations: npm run check:i18n. This one is advisory only, it never exits with a non-zero code, so treat a failure as something to read, not a gate that blocks a merge. There is no end to end suite, no dependency or secret scanning, and no approval gate. When you stand up hosted CI, wire in those same four commands and add a dependency scanner alongside them.
3	How is a release promoted from staging to production, and who can approve or execute it? There is no staging environment and no formal release ceremony. One Supabase project backs everything, declared in supabase/config.toml as project_id "tradeindrc", and the app reads NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY and SUPABASE_SERVICE_ROLE_KEY. One thing for you to fix: .env.example is stale, it still lists Firebase-era variables from an earlier iteration of the stack. Update it to the Supabase variables above before onboarding a new engineer. Work happens on main, and Vercel builds and deploys from main, so a push to main is a production release. Migrations reach production through supabase db push. Only one pull request has ever been merged, #1 at commit e203481. Small team, direct-to-main was the right tradeoff at this stage. Now that the site is going live, move to a dev branch for day-to-day work and main for production, so a release becomes a merge from dev into main. Put branch protection on main and name an approver before making that switch.
4	What is the rollback procedure, including database-migration rollback or forward-fix practice? The database is forward-fix only, and that is deliberate. All 41 migrations, 00001_initial_schema.sql through 00041_companies_premium_plan_strategic.sql, are append-only with no down sections, and the convention documented in CLAUDE.md is to never edit a shipped migration. Undoing a schema change means writing a new numbered migration, running supabase db push, and hand-editing src/lib/supabase/types.ts to match (supabase gen types is off limits, it overwrites the hand-authored type aliases at the top of that file). Backups and point-in-time recovery are not configured in the repository, that lives on the Supabase project settings side, not in code, and it is the first infrastructure decision to make before go-live.

NO.	QUESTION / INFORMATION PROVIDED
	On the application side, Vercel keeps every previous deployment, so the fastest rollback is promoting the last good deployment from the Vercel dashboard. The alternative is a git revert on main, which triggers a fresh build and deploy. Neither one undoes a schema change, the database only moves forward.
5	Where are release notes, incident records and change approvals maintained? There is no changelog, release notes, incident log or approval register, and no git tags. The record lives elsewhere instead: Git history, which follows Conventional Commits (feat(scope):, fix(scope):, chore(scope):), and the decision docs in docs/superpowers/specs/, most usefully 2026-05-17-master-roadmap-design.md, plus the per-folder CLAUDE.md files, which carry the rules and the reasoning behind them. Across 176 commits there are two merge commits and one GitHub pull request, #1 at commit e203481. So most changes have no formal approval trail. Once more than one person is committing, start tagging releases and running changes through pull requests to build that trail going forward.

7. Data, files and recovery

NO.	QUESTION / INFORMATION PROVIDED
1	Where are application files and media stored (AWS S3, Cloudinary, local server, etc.)? State bucket/container names and lifecycle policy. Everything a user uploads goes to Supabase Storage. Nothing is written to the Next.js filesystem. There are five buckets: • company-documents, private, holds verification documents (created in 00001_initial_schema.sql:503-506, re-asserted in 00011_verification_loop_fix.sql:111-113). • company-assets, public, written by src/lib/admin/asset-upload.ts with the names in src/constants/storage.ts. • product-images, public, used by src/components/dashboard/product-form.tsx:34. • company-media and company-brochures, both public (00018_company_rich_profile.sql:134-137), read through src/hooks/use-company-media.ts:25-26. Private documents are handed out as signed URLs that expire after SIGNED_URL_TTL_SECONDS = 600, defined at src/lib/verifications/actions.ts:37 and used at line 382. Static media sits in public/images, public/videos and public/seed-images. In next.config.ts the image optimiser sets dangerouslyAllowSVG: true, contentType: "attachment" and the image CSP default-src 'self'; script-src 'none'; sandbox;, with no remotePatterns configured. There are no lifecycle rules on the buckets. A file lives until something deletes it explicitly.
2	What data classifications exist (public, internal, confidential, personal data), and what privacy or retention requirements apply? There's no formal classification document, but the row-level security policies and bucket visibility already sort the data into three groups. • Public: companies, products, sectors, categories, hs_codes, tags, segments, services, institutions, content_items, page_content, faqs, help_articles, market_metrics, trade_series, sector_activity, price_points and price_series, plus everything in the public buckets. • Personal: profiles, the contact_email, contact_phone and address columns on companies, company_contacts, contact_submissions (which stores an ip inet column, 00017_page_content.sql:150 and 158), business_requests (submitter_id is nullable, 00028_business_requests_anonymous.sql:8), premium_requests, opportunity_responses, conversations, messages, conversation_participants, message_reports and analytics_events. • Confidential, admin-only through public.is_admin(): company_documents, kyc_individuals (00004_trust_center.sql:48), kyp_checks, verification_reviews, audit_log (00015) and the private bucket. The public privacy commitments live in the Privacy namespace of src/config/messages/en.json and fr.json and are rendered by src/app/[locale]/privacy/page.tsx. They promise that data is kept while the account is active, deleted on request, and that users hold the rights granted by DRC Law No. 20/017 of 25 November 2020, exercised via privacy@tradeindrc.com, with a clause on sharing with the Ministry.

NO.	QUESTION / INFORMATION PROVIDED
	Retention and erasure aren't enforced by any job. The only automatic cleanup is ON DELETE CASCADE (00001:43 and 00001:149).
3	How often are databases and file stores backed up? State backup location, retention period, encryption and restoration owner. This project doesn't back anything up itself, and deployment doesn't touch backups either. We push to main, Vercel's GitHub integration builds and deploys it, and branches get preview URLs. That's the whole release path. Whatever backups exist come from the managed Supabase project, ref <code>gwqjwxovcqfrvrebzqi</code>, named in the git-ignored <code>.mcp.json</code>. It's on the free plan today, which means no point-in-time recovery and only minimal retention, so there's no real backup position to hand over as things stand. Frequency, retention, point-in-time recovery and encryption at rest are all dashboard settings under Project, Database, Backups, and none of them are recorded in code. There are two ways forward for the receiving team. Moving to a paid Supabase plan turns on daily backups and point-in-time recovery, and Supabase's privacy terms are acceptable on a paid account. Supabase is also open source, so the whole stack can instead be self-hosted on infrastructure the Ministry controls, which is the stronger answer to a government-approved-provider requirement. No restoration owner is documented. Today, recovery falls to whoever holds the Supabase owner account and the <code>SUPABASE_SERVICE_ROLE_KEY</code>. Before go-live, the receiving team should pick between the paid plan and self-hosting, name a restoration owner and a deputy, add off-platform <code>pg_dump</code> dumps and Storage copies, and rotate the plaintext Supabase token sitting in <code>.mcp.json</code> — that rotation is standard handover hygiene given the token is in a local, git-ignored file.
4	When was the last successful restoration test, and what recovery time/recovery point objectives are targeted? A restore has never been tested. There's no restore runbook and no recovery time or recovery point objective anywhere in the project, in <code>README.md</code>, in the <code>CLAUDE.md</code> files or in <code>docs/</code>. Recovery currently rests on a free-plan Supabase project with no point-in-time recovery, so as things stand a serious data loss would not be fully recoverable. It's worth being clear about how quality is handled here, because it shapes what "tested" means in this repo. Checks aren't run by a hosted CI server. They're driven by the agentic workflow and the convention layer the repository carries: a root <code>CLAUDE.md</code>, seven folder-level <code>CLAUDE.md</code> files and <code>docs/MOTION.md</code> define the architecture rules, the Supabase client rules, the migration rules and the motion rules. Coding agents work against those files and run <code>npm run lint</code>, <code>npm test</code> (Vitest, 13 test files, 58 cases), <code>npm run build</code> and <code>npm run check:18n</code> before code lands. That's a deliberate choice for a project with one engineer committing; it covers the code path well, but it doesn't cover infrastructure recovery, which nobody has rehearsed. Once more than one person is committing, a hosted CI pipeline is the natural next step. For the receiving team: agree an RTO and RPO with the Ministry, then rehearse a restore into a scratch project by re-pointing <code>NEXT_PUBLIC_SUPABASE_URL</code>, <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> and <code>SUPABASE_SERVICE_ROLE_KEY</code>, running <code>npm run build</code>, and smoke-testing login, registration and the verification queue. Storage restore needs its own rehearsal, and both results should be written down.
5	What data migrations, imports, exports or master-data dependencies must be understood before takeover? The schema is 41 numbered SQL files in <code>supabase/migrations/</code>, from <code>00001_initial_schema.sql</code> to <code>00041_companies_premium_plan_strategic.sql</code>, applied with <code>supabase db push</code>. Two rules govern it. Push every new migration to production straight away. And never run <code>supabase gen types: src/lib/supabase/types.ts</code> is maintained by hand, on purpose, because the generator wipes the aliases <code>VerificationTier</code> (line 10), <code>BusinessRequestStatus</code> (line 94) and <code>PremiumPlan</code> (line 101). Edit that file by hand in the same commit as the migration. Some master data arrives already seeded. <code>00006_marketplace_segments.sql:31</code> inserts 8 segments and <code>00034_marketplace_investors_segment.sql:25</code> adds an investors segment with <code>name_es</code>, <code>name_tr</code> and <code>name_zh</code> columns. <code>00033_market_intelligence_seed.sql</code> loads 8 <code>market_metrics</code> rows, 27 <code>trade_series</code> rows and 48 <code>sector_activity</code> rows, all static figures the Ministry has to refresh. Storage buckets are created in 00001, 00011 and 00018. The rest is entered by hand through the admin UI: sectors, categories, <code>hs_codes</code> and tags (<code>00014_taxonomy_hs_tags.sql</code> contains no <code>INSERTs</code>), institutions (00029), <code>site_settings</code>, <code>page_content</code> and <code>carousel_slides</code>. Provinces aren't in the database at all: the 26 entries in <code>DRC_PROVINCES</code> in <code>src/config/provinces.ts</code> are matched against <code>companies.province</code>.

NO.	QUESTION / INFORMATION PROVIDED
	There are three import paths: 1. Admins upload a price CSV through <code>src/app/[locale]/admin/data-hub/prices/points-uploader.tsx</code>, parsed by <code>parsePriceCsv</code> in <code>src/lib/data-hub/csv.ts</code> and written to <code>price_points</code>. 2. <code>npm run seed:logos</code> runs <code>scripts/seed-images/build-logos.ts</code>. 3. <code>npm run seed:images</code> runs <code>scripts/seed-images/fetch.ts</code>, which needs <code>UNSPLASH_ACCESS_KEY</code> and <code>PEXELS_API_KEY</code>, reads <code>lib/sources.ts</code> and <code>manifest.json</code>, and records credits in <code>public/seed-images/ATTRIBUTIONS.md</code>. There is no export path: no CSV or JSON endpoint, no download UI, no data subject access request export. That's a gap the receiving team needs to close, particularly for DSAR compliance under the privacy commitments in section 2 above.

8. Security, monitoring and operations

NO.	QUESTION / INFORMATION PROVIDED
1	Where are configuration variables and secrets managed (secret manager/password manager), and who administers access? I didn't stand up a dedicated secret manager for this. Locally, secrets live in a git-ignored <code>.env</code> at the repo root (only <code>.env.example</code> is tracked); in production they live in the Vercel project's environment variables, which is the de facto production secret store. Nine variables are in use: <code>NEXT_PUBLIC_SUPABASE_URL</code>, <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>, <code>SUPABASE_SERVICE_ROLE_KEY</code>, <code>NEXT_PUBLIC_SITE_URL</code>, <code>CAPTCHA_SECRET_KEY</code>, <code>NEXT_PUBLIC_CAPTCHA_SITE_KEY</code>, <code>UNSPLASH_ACCESS_KEY</code>, <code>PEXELS_API_KEY</code> and <code>NODE_ENV</code>. Two things for the receiving team to set up at handover. First, <code>.env.example</code> still describes the old Firebase stack and needs updating to match what's actually deployed. Second, nobody currently owns these secrets on paper: there's no rotation schedule and no access matrix. The new owner should stand up a secret store and name who owns the Supabase project (ref <code>gwqjwxovcqfrvfrebzi</code>).
2	What security controls are enabled: MFA, least-privilege roles, audit logging, WAF, vulnerability scanning, dependency scanning and encryption? What's in place today: • Route authorization. <code>src/proxy.ts</code> is the Next 16 middleware (it replaced the old <code>src/middleware.ts</code>) and guards the <code>PROTECTED_ROUTES</code> and <code>ADMIN_ROUTES</code> lists in <code>src/constants/routes.ts</code> using <code>isAdmin()</code> from <code>src/constants/roles.ts</code>, which reads <code>profiles.staff_role</code>. The admin area checks again server-side in <code>admin/layout.tsx</code> via <code>requireAdmin()</code> (<code>src/lib/auth/require-admin.ts</code>), and the database has its own <code>is_admin()</code> SQL function from migrations 00002 and 00013. • Row-level security is on for all 46 tables, and every table carries at least one policy. Storage bucket policies come from migration 00001. • <code>SUPABASE_SERVICE_ROLE_KEY</code> is server-only. It's read through <code>src/lib/supabase/admin.ts</code>, which 23 server-side files import and no client file touches. • Audit logging writes to <code>public.audit_log</code> (migration 00015). Admins can read it, nothing can write to it through the API, and rows are inserted from <code>src/lib/admin/users-actions.ts</code> and <code>admin/settings/actions.ts</code>. • Anti-abuse is Turnstile CAPTCHA (<code>src/lib/messaging/captcha.ts</code>) plus rate limits in <code>src/lib/messaging/actions.ts</code>, capped at 20 messages and 10 threads per hour, failing closed. • Encryption is whatever Supabase and the host provide by default. What I'd put on the new owner's roadmap next: multi-factor authentication, a WAF, vulnerability scanning, dependency scanning and static analysis. None of these were in scope for this build.
3	Which monitoring, logging, error-tracking and uptime tools are used, and where are the dashboards and alerts? There's no monitoring stack. No error tracking is wired in, and no uptime probes are configured. This wasn't part of the build scope.

NO.	QUESTION / INFORMATION PROVIDED
	The only instrumentation is first-party product analytics: an <code>httpOnly visitor_id</code> cookie set in <code>src/proxy.ts</code>, the <code>analytics_events</code> table from migration 00001, and the writers in <code>src/lib/analytics/track-event.ts</code> and <code>src/lib/search/analytics.ts</code>. That data feeds the admin screens, so it's a product feature, not an operations tool. Today, errors surface only in host runtime logs and the Supabase dashboard logs. Both are retention-limited and neither one pages anyone when something breaks. Error tracking, uptime probes and a log drain are the three things I'd add first before go-live.
4	What alerts are considered critical, who receives them and what is the escalation/runbook process? None of this exists yet: nothing is scheduled, no on-call rota or escalation policy is written down, and no person is named as owner. That's work for the new owner to set up. These are the alerts I'd define first, in priority order: • Site down, or a rising 5xx rate. The whole app is one Next.js deployment, so this means a total outage. • Supabase unavailable or over quota. It holds the database, auth and storage, so this is also a total outage. • Authentication failures at scale. <code>requireAdmin()</code> returns a 500 when the <code>profiles</code> fetch fails, which locks every admin out of <code>/admin</code>. • Unusual writes to <code>public.audit_log</code>, or changes to <code>profiles.staff_role</code>. • Spikes in CAPTCHA rejections or rate limiting in <code>src/lib/messaging/actions.ts</code>. Each one needs a named recipient and a one-page runbook.
5	Are there known security findings, technical debts, single points of failure or services approaching expiry/end of support? Ten items, roughly in order of urgency: 1. <code>.mcp.json</code> stores a live Supabase access token in plaintext as a <code>--access-token</code> argument, alongside <code>--project-ref gwqjwxovcqfrvrbzqi</code>. The file is git-ignored and was never committed, but the token sits unencrypted on disk and shows up in process arguments. This is normal handover hygiene: rotate it at handover. 2. Supabase is a single point of failure. It holds the database, auth, storage and RLS, with no replica, no failover and no documented backup or restore procedure. The project is still on the free plan, so there's no point-in-time recovery either. This is the first infrastructure decision the new owner should make: move to a paid plan, or self-host Supabase, before go-live. 3. Quality checks run through an agentic workflow rather than a hosted CI server. That's a deliberate choice for a one-engineer build. The repo carries a convention layer instead: a root <code>CLAUDE.md</code> plus seven folder-level <code>CLAUDE.md</code> files and <code>docs/MOTION.md</code>, spelling out the architecture rules, the Supabase client rules, the migration rules and the motion rules. Coding agents work against those files and run <code>npm run lint</code>, <code>npm test</code> (Vitest, 13 test files, 58 cases), <code>npm run build</code> and <code>npm run check:i18n</code> before code lands. The checks run every time, they're just driven by the agent and the conventions rather than a pipeline. Once more than one person is committing, wiring those same four commands into a hosted pipeline is the natural next step, and I'd recommend doing it at that point. 4. No error tracking, monitoring or uptime checks, as covered in question 3. Same recommendation applies. 5. Two lockfiles are checked in, <code>package-lock.json</code> and <code>pnpm-lock.yaml</code>. I'd standardize on pnpm: <code>pnpm-lock.yaml</code> is the one that's been kept current, and <code>package-lock.json</code> is a legacy artifact the new owner can remove once the team is on pnpm. 6. <code>.env.example</code> is stale and still lists the old Firebase stack. Update it to match the current Supabase-based env vars listed in question 1. 7. <code>next.config.ts:11</code> sets <code>dangerouslyAllowSVG: true</code>. This is deliberate, and the image CSP on line 13 is the mitigation for it. 8. Next 16.1.1, React 19.2.3 and Tailwind v4 are recent majors, so nothing is near end of support, but there's no automated update path. I'd add Dependabot or Renovate. 9. <code>src/lib/supabase/CLAUDE.md:9</code> still points at the deleted <code>src/middleware.ts</code> instead of <code>src/proxy.ts</code>. Worth a quick fix so the convention layer stays accurate for whoever reads it next. 10. No production domain is registered. The code assumes <code>tradeindrc.com</code>, which nobody has bought yet, so the site answers only on its Vercel URL until the client buys the domain. On deployment itself: this runs on Vercel's GitHub integration. Every commit to main builds and deploys automatically, and branches get their own preview URLs, so nothing is pushed by hand.

9. Quality assurance and known issues

NO.	QUESTION / INFORMATION PROVIDED
1	What test suites exist, how are they executed, and what coverage or quality targets are tracked? Automated testing is a unit and component suite on Vitest 4.1.6. vitest.config.ts uses jsdom, global test APIs, a vitest.setup.ts file, and maps @ to ./src. There are 13 test files under src/ with 58 cases; the biggest are register/market/registration-profile.test.ts (12 cases), data-hub/csv.test.ts (11) and url/external-href.test.ts (7). • npm test runs vitest run, and npm run test:watch runs it in watch mode. • The other checks are npm run lint, npm run build and npm run check:i18n. Those four commands are the quality gate. I ran them through an agentic workflow rather than a hosted pipeline, because one engineer was committing and the repo carries a strong convention layer for an agent to work against: a root CLAUDE.md, seven folder-level CLAUDE.md files, and docs/MOTION.md, spelling out the architecture rules, the Supabase client rules, the migration rules and the motion rules. An agent reads those, makes the change, and runs lint, test, build and check:i18n before the code lands. The checks run on every change, enforced by the agent and the conventions rather than by a server. There's no coverage provider or threshold configured, and no end-to-end or visual regression tests. Once more than one person is committing, stand up a hosted CI pipeline running these same four commands against every PR — that's the natural next step for a multi-engineer team.
2	What browsers, devices and accessibility requirements are supported? • Browsers: there's no browserslist and no target override in next.config.ts, so the app targets the Next.js 16.1.1 default of modern evergreen browsers (Chrome, Edge, Safari, Firefox). Legacy browsers are untested. Linting uses eslint-config-next/core-web-vitals, but Core Web Vitals aren't measured. • Devices: layouts are responsive through Tailwind v4 and shadcn/ui, with a mobile nav in src/components/layout/mega-menu.tsx. There's no device matrix and no cross-browser testing service in the loop. • Accessibility is built in at several points: a reduced-motion block at src/app/globals.css lines 186-193, useReducedMotion() in 14 files, the rules in docs/MOTION.md sections 4 and 7, Radix primitives, 77 aria-label attributes, 8 sr-only elements, and <html lang={locale}> at src/app/[locale]/layout.tsx line 92. • A WCAG 2.1 AA conformance statement, an automated accessibility scan (axe or pa11y) and a skip-to-content link are not in place yet. If accessibility conformance becomes a contractual requirement, that's the order I'd add them in.
3	What are the current defects, known limitations, performance bottlenecks and priority backlog items? • Quality gating is agent-driven, not pipeline-driven (see the answer above). Lint, test, build and check:i18n run before code lands, but nothing enforces that server-side yet. • Translations cover 5 locales (en, fr, tr, es, zh) with 3,534 keys each, but scripts/check-i18n.js only compares English against French, so TR, ES and ZH are unverified. EN and FR are clean apart from 208 English keys that look unused. • A few docs are stale: docs/implementation/todo.md and docs/short-term-plan/CONTINUITY.md still describe Firebase and Next 15, the root CLAUDE.md points at src/middleware.ts when the real file is src/proxy.ts, and docs/error-solving/understood-errors.md is empty. Worth a cleanup pass before anyone relies on them. • Roadmap work still open in docs/superpowers/specs/2026-05-17-master-roadmap-design.md: slice S6 Data Hub is partial (prices and reports exist, the legal library doesn't), and S8 and S9 are untouched. Payments and escrow, and realtime notifications, were scoped out as non-goals for this build, not deferred by accident. • There's no transactional email dependency; the app relies on Supabase Auth mail for that, by design. • Test depth is thin by intent, not oversight, for a single-engineer build: 58 unit tests, no end-to-end tests, no profiling or Lighthouse budget, and no error tracking wired in. These are the natural next investments as the team and traffic grow.
4	What business scenarios must be demonstrated to confirm that the transfer is successful? 1. Switch language between /en and /fr using language-switcher.tsx, and confirm npm run check:i18n reports no problems. 2. Sign up and log in at /[locale]/(auth)/signup and /login, and confirm src/proxy.ts sends an anonymous visit to / dashboard onward to /[locale]/login?redirect=...

NO.	QUESTION / INFORMATION PROVIDED
	3. Register a company at <code>/[locale]/register-company</code>, checking that the Congolese and international paths ask for different things. 4. Approve or reject a company as an admin at <code>/[locale]/admin/verifications</code> (<code>src/lib/verifications/actions.ts</code> and <code>decision-panel.tsx</code>), then see the badge appear on <code>/companies/[id]</code> and <code>/trust/[companySlug]</code>. 5. Create a product at <code>/[locale]/dashboard/products/new</code> and find it on <code>/products</code> and <code>/market/[segment]</code>. 6. Send an RFQ from <code>/[locale]/request</code> or <code>/rfq</code> and read it at <code>/[locale]/admin/requests</code>. 7. Request premium from <code>/[locale]/pricing</code> and approve it at <code>/[locale]/admin/requests/premium</code>, which calls <code>approvePremiumRequest</code>. 8. Run <code>npm ci</code> && <code>npm run lint</code> && <code>npm test</code> && <code>npm run build</code> green against the handed-over Supabase project. 9. Push a commit to main and confirm Vercel's GitHub integration builds and deploys it on its own. Open a branch and confirm you get a preview URL for it.

10. Handover acceptance checklist

Acceptance item	Status	Evidence / link
Receiving team has read access to all repositories and documentation.	☐ Complete ☒ Pending ☐ N/A	The repository is <code>github.com/its-meseba/trade-in-drc-com-web-app</code> . Access is granted per named GitHub account, after which the repository should be transferred to the client. The documentation sits inside the repository. See answers 2.1 and 2.5.
Receiving team has appropriate, least-privilege access to hosting, DNS, monitoring, CI/CD and secret-management systems.	☐ Complete ☒ Pending ☐ N/A	This means the Vercel project that hosts the site and the Supabase project named <code>tradeindrc</code> . There's no monitoring stack and no domain registrar account yet, so those two have nothing to hand over. Credentials go through the password manager, never through this document.
A clean local setup and a staging deployment were successfully demonstrated.	☐ Complete ☒ Pending ☐ N/A	Answer 2.3 has the setup steps: clone, install, fill in <code>.env</code> , run <code>supabase db push</code> over the 41 migrations, then <code>npm run dev</code> . There's no staging project. Vercel gives every branch a preview URL, which is the closest thing we have until someone builds one.
Production deployment and rollback procedures were reviewed and tested or simulated.	☐ Complete ☒ Pending ☐ N/A	Vercel builds and deploys every push to main, and rollback is promoting a previous deployment from the Vercel dashboard. Database migrations only run forwards. See answers 6.1, 6.3 and 6.4. A rollback should be rehearsed before acceptance.
Backup location and restoration procedure were verified.	☐ Complete ☒ Pending ☐ N/A	The Supabase project is on the free plan, so there is no point-in-time recovery and only minimal retention. Either move to a paid plan or self-host Supabase before go-live, then name a restoration owner and rehearse a restore. Nothing has been tested. See answers 7.3 and 7.4.

Acceptance item	Status	Evidence / link
Critical third-party integrations, renewal responsibilities and support contacts were verified.	☐ Complete ☒ Pending ☐ N/A	The list of connected services is in answers 4.1 and 4.2. Who owns each account, and when each one renews, is settled at sign-off and recorded there.
Known risks, technical debt and open actions were acknowledged.	☐ Complete ☒ Pending ☐ N/A	The open items are listed in answers 8.5 and 9.3. The Supabase token kept in the local .mcp.json file needs rotating. The Supabase project is still on the free plan, with no backups worth the name. No production domain has been bought. There is no CI and no monitoring, two lockfiles are committed, and .env.example is out of date.

Sign-off

Signing below confirms that both parties have reviewed this handover. Anything still outstanding is recorded in the open items list at answer 8.5.

Transferring representative Name and role: Mehmet Semih Babacan Signature / date:	Receiving representative Name and role: Signature / date:
---	---